

Lab 6 : Design of a Simple General-Purpose Processor

COE 328 : Digital Systems

Professor Reza Sedaghat

Teaching Assistant: Nahid Sahel Gozin

Pushpinder Singh

501294986

Table of Contents

Introduction: **3**
Components: **3**
Latch 1 and 2: 3
Description: 3
Truth Table: 3
Block Diagram: 4
Waveform: 4
4x16 Decoder: 4
Description: 4
Truth Table: 4
Block Diagram: 5
Waveform: 6
Finite State Machine (FSM): 6
Description: 6
State Table: 7
Block Diagram: 7
Waveform: 7
ALU_1 for Problem Set 1 of the Lab 6 procedure: **8**
Description: 8
Block Diagram: 8
Circuit Schematics for Problem Set 1: 8
Inputs and Outputs of the ALU: 8
Table of function related to unique Microcode's generated by decoder for ALU_1: 9
Calculations: 9
Waveform: 10
ALU_2 for Problem Set 2 of the Lab 6 procedure: **11**
Block Diagram: 11
Circuit Schematics for Problem Set 2: 11
Table of function related to unique Microcode's generated by decoder for ALU_2: 12
Calculations: 12
Waveform: 13
ALU_3 for Problem Set 3 of the Lab 6 procedure: **14**
Description: 14
Block Diagram: 14
Circuit Schematics for Problem Set 3: 14
Assigned Problem from Problem Set 3: 15
Table of function related to unique Microcode's generated by decoder for ALU_3: 15
Waveform: 16
Conclusion: **17**

Introduction:

This lab aims to design and implement the hardware of a custom arithmetic logic unit (ALU) which is coordinated by a custom finite state machine (FSM) on an Altera DE2 FPGA board. The main objective of this lab is to integrate various sub-components to perform synchronized data routing and data manipulation. The FSM type assigned for this lab was a Mealy Machine that dictates the sequence of operations based on clock cycles and manual toggle switch inputs. As the FSM is clocked, it outputs a 4-bit state sequence which is directly fed to the 4x16 decoder. The decoded translates the 4 bit state input into a precise 16-bit OP code that is used to instruct ALU with the type of function to perform. Alongside the control unit and ALU, two 8 bit registers are used to store the input data A and B and feed directly to the ALU in the next cycle. Then, the ALU processes the inputs based on the OP code from the control unit. The 8 bit arithmetic result from the ALU is split into 2 parts and routed into 2 seven segment displays. Ultimately, this system demonstrates the integration of sequential control routing and combinational arithmetic logic to execute microcode instructions in physical hardware.

Components:

Latch 1 and 2:

Description:

Latch 1 and 2 serve as 8-bit synchronous registers designed to capture and hold the user input A and B. These registers operate on the positive-edge of the clock cycle as they are designed to be positive-edge triggered. It grabs the input from the physical toggle switches and feeds it to the ALU. It prevents corruption in ALU's arithmetic process by safely holding the data between the clock cycles. It utilizes low-active reset to reset the values to all zeroes when required.

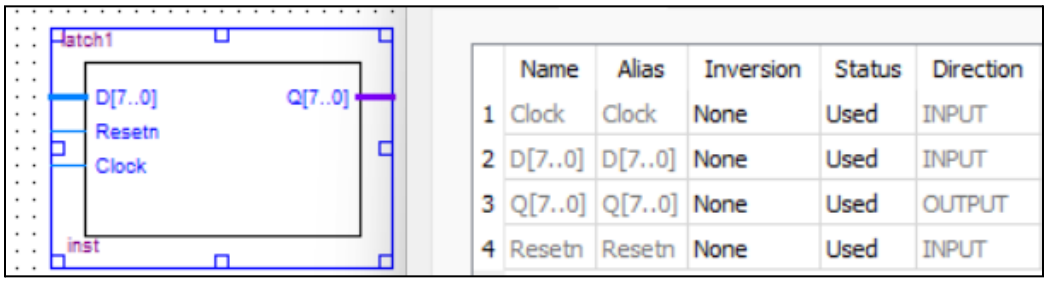
Dataset A and B are based on the last 4 digits of my student ID. My Student Id is 501294986, so A = 49 and B=86 in Hexadecimal format, which translates to A = 01001001 and B = 10000110 in binary.

Truth Table:

Reset	Clock	Input	Output	Description
0	**Up or Down**	**Any 8 bit input**	**00000000**	**Output reset to zero when Reset = 1**
1	**UP**	**Q(8 bit)**	**Q(8 bit)**	**Same input and output when positive triggered.**
1	**UP**	**X(8 bit)**	**X(8 bit)**	**Same input and output when positive triggered.**

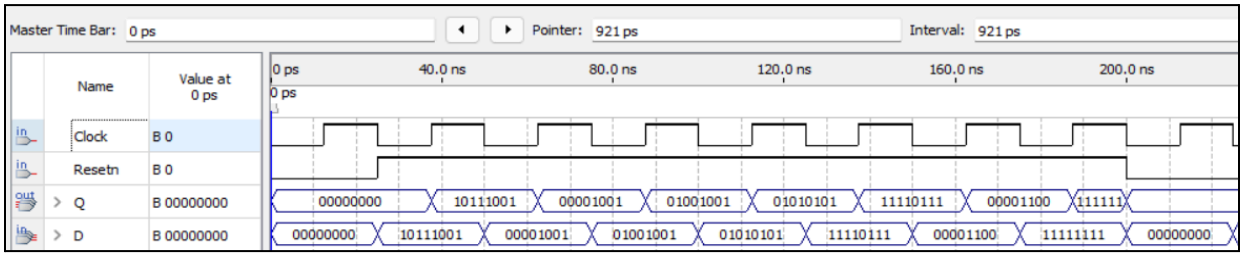
1	**0 or DOWN**	**Z(8 bit)**	**Previous Input**	**Holds previous input until the next positive edge is triggered.**

Block Diagram:



	Name	Alias	Inversion	Status	Direction
1	Clock	Clock	None	Used	INPUT
2	D[7..0]	D[7..0]	None	Used	INPUT
3	Q[7..0]	Q[7..0]	None	Used	OUTPUT
4	Resetn	Resetn	None	Used	INPUT

Waveform:



We can see here in the waveform that when Reset is 1, the output follows the input at every rising edge of the clock and it retains the previous state when the Reset is set to 0.

4x16 Decoder:

Description:

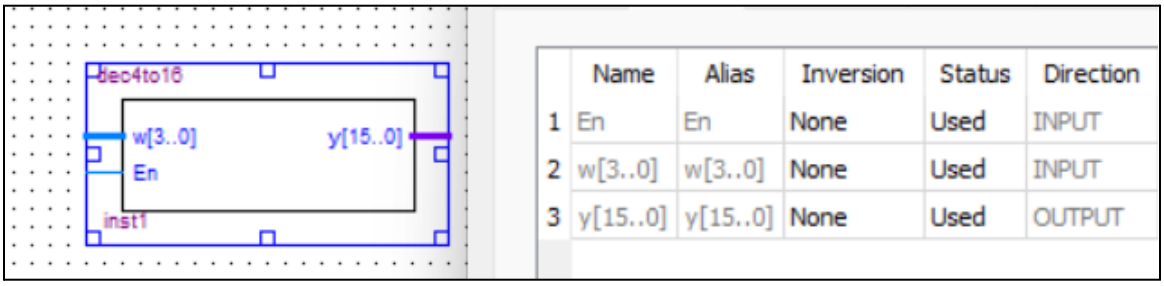
This is a combinational logic component that translates the 4-bit state input from the FSM into a 16-bit OP code. It incorporates an Enable switch, which when turned ON (Enable=1), the decoder takes the 4-bit input from the FSM and translates it into 16-bit OP code and feeds directly to the ALU unit. The unique 16 bit OP code is only one hot operation code which means only one output of the OP code is 1 and the rest of the outputs are zero, for example the output of the decoder for 0th state is 0000000000000001, which corresponds to function number 1 in ALU.

Truth Table:

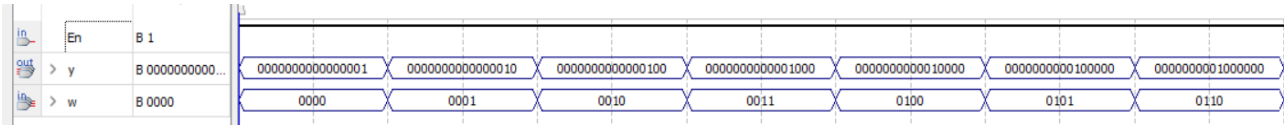
Enable	Input (4-bit State from FSM)	Output (16-bit OP code fed directly to the ALU)
0	**xxxx**	**0000000000000000**

1	0000	0000000000000001
1	0001	0000000000000010
1	0010	0000000000000100
1	0011	0000000000001000
1	0100	0000000000010000
1	0101	0000000000100000
1	0110	0000000001000000
1	0111	0000000010000000
1	1000	0000000100000000
1	1001	0000001000000000
1	1010	0000010000000000
1	1011	0000100000000000
1	1100	0001000000000000
1	1101	0010000000000000
1	1110	0100000000000000
1	1111	1000000000000000

Block Diagram:



Waveform:

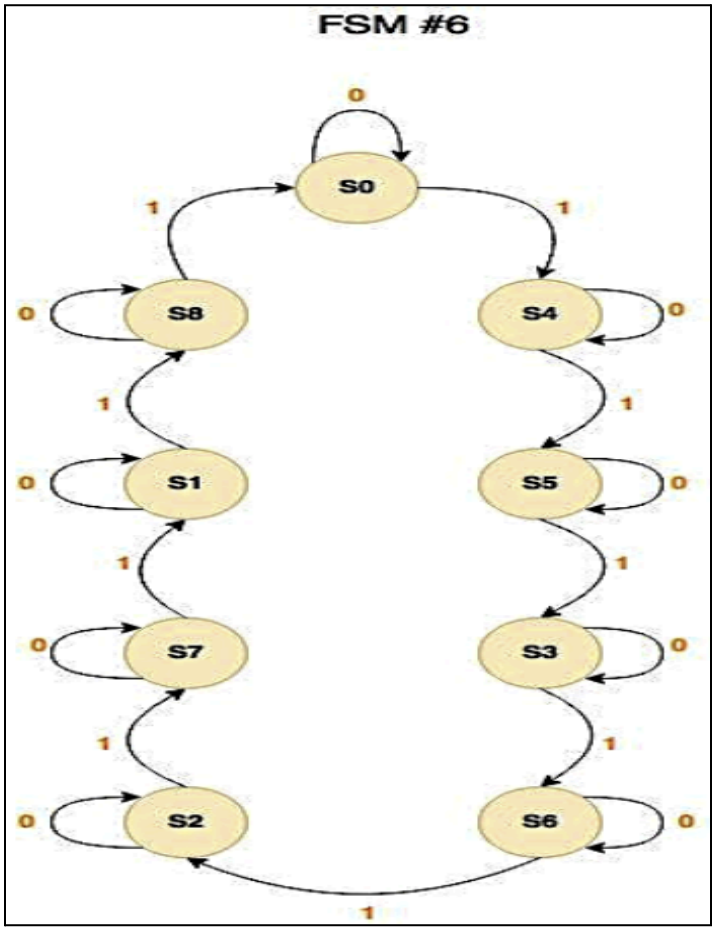


When Enable is 1, the decoder takes the state number from the FSM and outputs a 16 bit one hot operation code to the ALU. This waveform perfectly matches with the values in the truth table of 4x16 decoder.

Finite State Machine (FSM):

Description:

The FSM type assigned to be implemented is a Mealy-type Finite State Machine that controls the sequence of operations for the entire datapath. It transitions through a custom sequence of states that is triggered by the positive edge of the clock and the data input. The sequence of states assigned to me was FSM 6 as shown in figure below. Since it is a Mealy machine, the output of the FSM is dependent upon the data input and the current state input. At each valid state, the FSM outputs a 4-bit student ID that is directed to another seven segment display and a 4-bit state number that drives the decoder, effectively acting as a central brain responsible for ALU calculations.

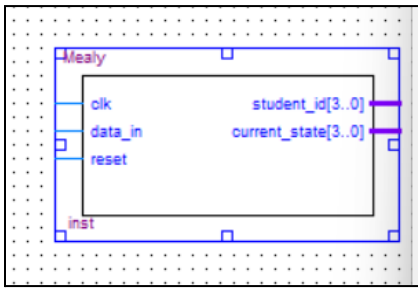


State Table:

State table:

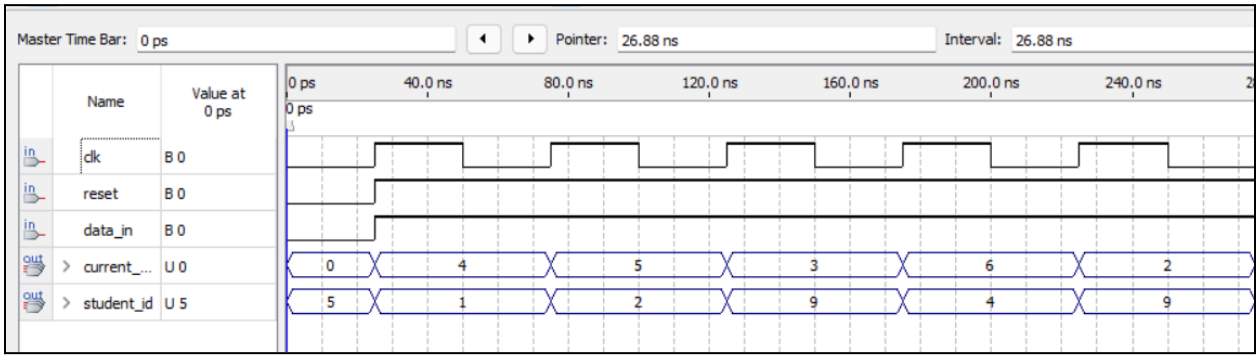
Present state	Next State w=0	Next State w=1	Output w=0	Output w=1
S_0	S_0	S_4	5	0
S_1	S_1	S_8	8	6
S_2	S_2	S_7	4	9
S_3	S_3	S_6	2	9
S_4	S_4	S_5	0	1
S_5	S_5	S_3	1	2
S_6	S_6	S_2	9	4
S_7	S_7	S_1	9	8
S_8	S_8	S_0	6	5
E				

Block Diagram:



	Name	Alias	Inversion	Status	Direction
1	clk	clk	None	Used	INPUT
2	current_state[3..0]	current_state[3..0]	None	Used	OUTPUT
3	data_in	data_in	None	Used	INPUT
4	reset	reset	None	Used	INPUT
5	student_id[3..0]	student_id[3..0]	None	Used	OUTPUT

Waveform:



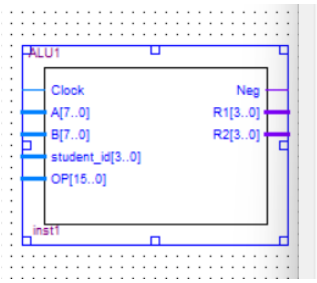
As we can see here, when data_in is 0, the student_id is 5 and the current state is 0, which is the first row in the state table. When the next clock cycle occurs and if data_in is set to 1, the student_id is 1 and the current state is 4, which exactly matches with the 5th row of the state table.

ALU_1 for Problem Set 1 of the Lab 6 procedure:

Description:

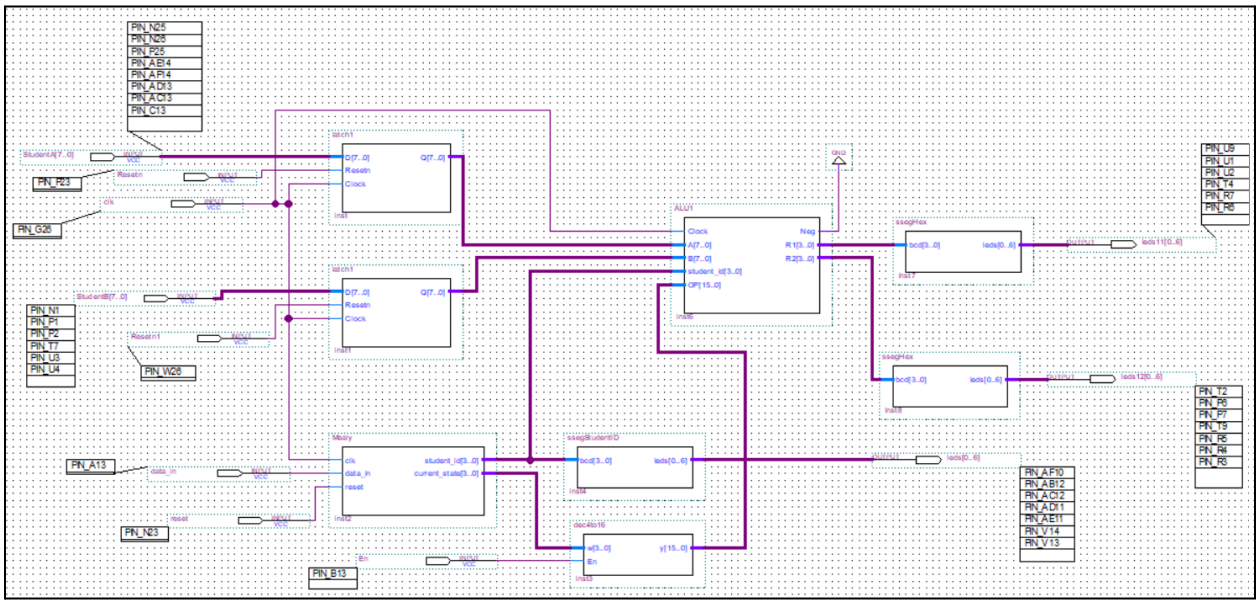
ALU 1 is a custom synchronous Arithmetic Logic Unit designed to perform a specific set of 9 arithmetic and Boolean logic operations. It processes two 8-bit inputs A and B captured from latch 1 and 2, executing calculations on the positive edge of the clock cycle. The specific operation is dictated by a 16-bit OP code received from the 4x16 decoder from the control unit. To support the verification, ALU splits the 8-bit computational output into two separate 4-bit outputs, allowing the data to be routed to two separate seven segment displays.

Block Diagram:



	Name	Alias	Inversion	Status	Direction
1	A[7..0]	A[7..0]	None	Used	INPUT
2	B[7..0]	B[7..0]	None	Used	INPUT
3	Clock	Clock	None	Used	INPUT
4	Neg	Neg	None	Used	OUTPUT
5	OP[15..0]	OP[15..0]	None	Used	INPUT
6	R1[3..0]	R1[3..0]	None	Used	OUTPUT
7	R2[3..0]	R2[3..0]	None	Used	OUTPUT
8	student_id[3..0]	student_id[3..0]	None	Used	INPUT

Circuit Schematics for Problem Set 1:



Inputs and Outputs of the ALU:

- *Clock (Input):* Global signal responsible for ALU calculations to occur on the positive edge cycle.
- *A & B (Inputs):* The 8-bit dataset supplied by Latch 1 and 2.
- *Student_Id (Input):* 4-bit signal routed from FSM to the ALU.
- *OP (Input):* 16-bit operation selector generated by 4x16 decoder. The position of "1" in the OP code tells the ALU which operation to perform.
- *R2 (Output):* The upper 4 bits (7 down to 4) of the final computation result.

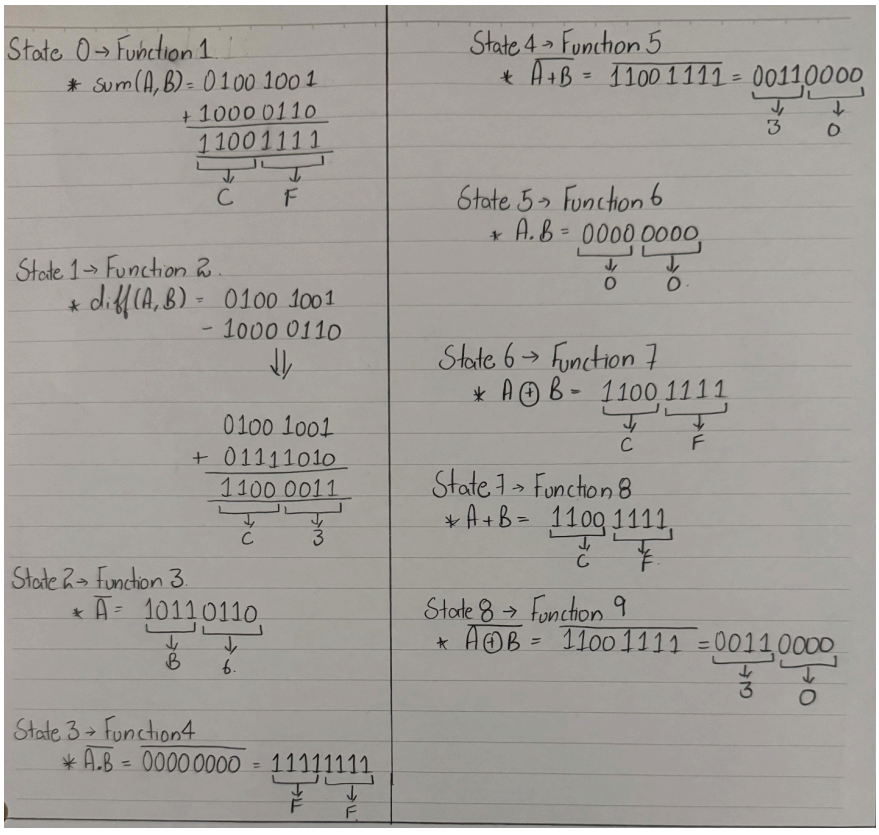
- *R1 (Output):* The lower 4 bits (3 down to 0) of the final computation result.

Table of function related to unique Microcode's generated by decoder for ALU_1:

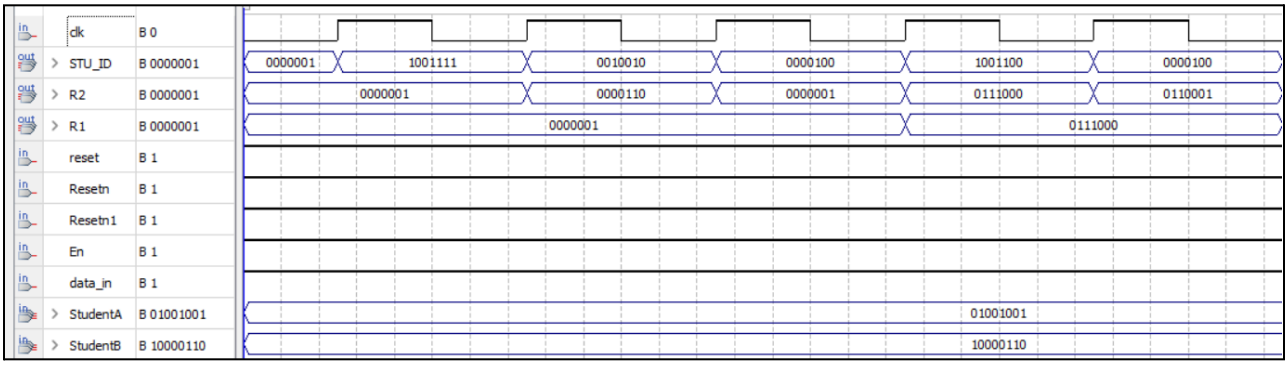
Function #	Microcode	Boolean Operation / Function
1	0000000000000001	sum(A, B)
2	0000000000000010	diff(A, B)
3	0000000000000100	$\overline{A}$
4	0000000000001000	$\overline{A \cdot B}$
5	0000000000010000	$\overline{A + B}$
6	0000000000100000	A · B
7	0000000001000000	A ⊕ B
8	0000000010000000	A + B
9	0000000100000000	$\overline{A \oplus B}$

Calculations:

The following figure shows the hand calculations for each operation or function for Problem set 1:

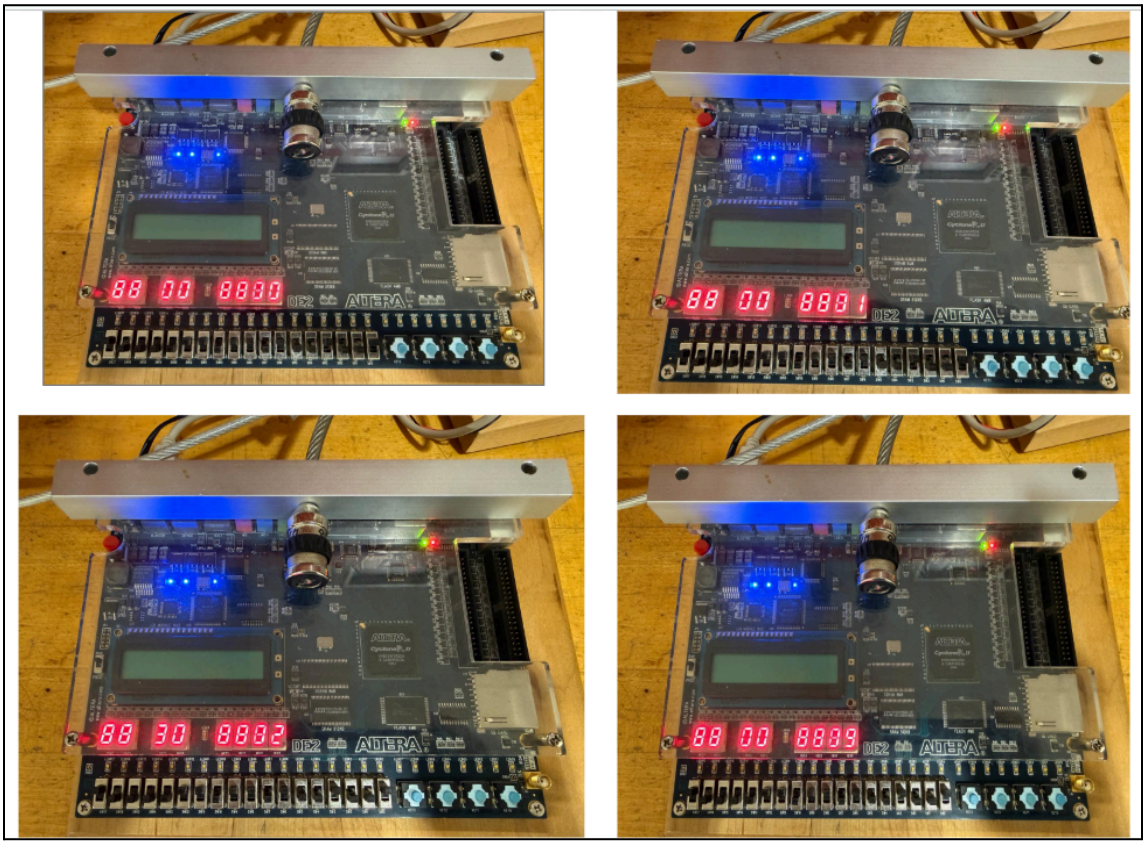


Waveform:



We can see here that the output R2 and R1 changes at every clock cycle. We set data_in to 1, in order to verify the waveform with the calculated results. When the first clock hits, the latches take the input and feed directly to the ALU. At the same time, the FSM outputs the state number to the decoder and the decoder converts it to OP code and feeds it to the ALU. Now, when the next cycle hits, ALU takes the input A and B from the latches, and OP code from the decoder and performs a specific function and outputs the results to R1 and R2. We can see in the waveform that when studentId is 1 and data_in is 1, the state number is 4, so when the next cycle clock hits, ALU should perform the calculator assigned to the OP code related to the state number 4. By hand calculation, we know that the result of R1 and R1 should be 3 0 which exactly matches in the waveform given above.

Verification of results on Altera DE2 FPGA board:

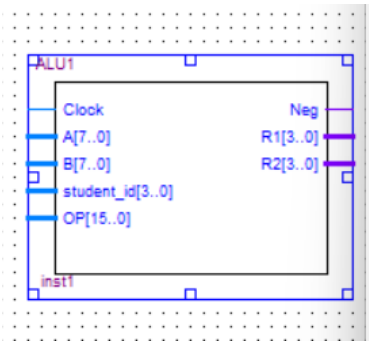


ALU_2 for Problem Set 2 of the Lab 6 procedure:

Description:

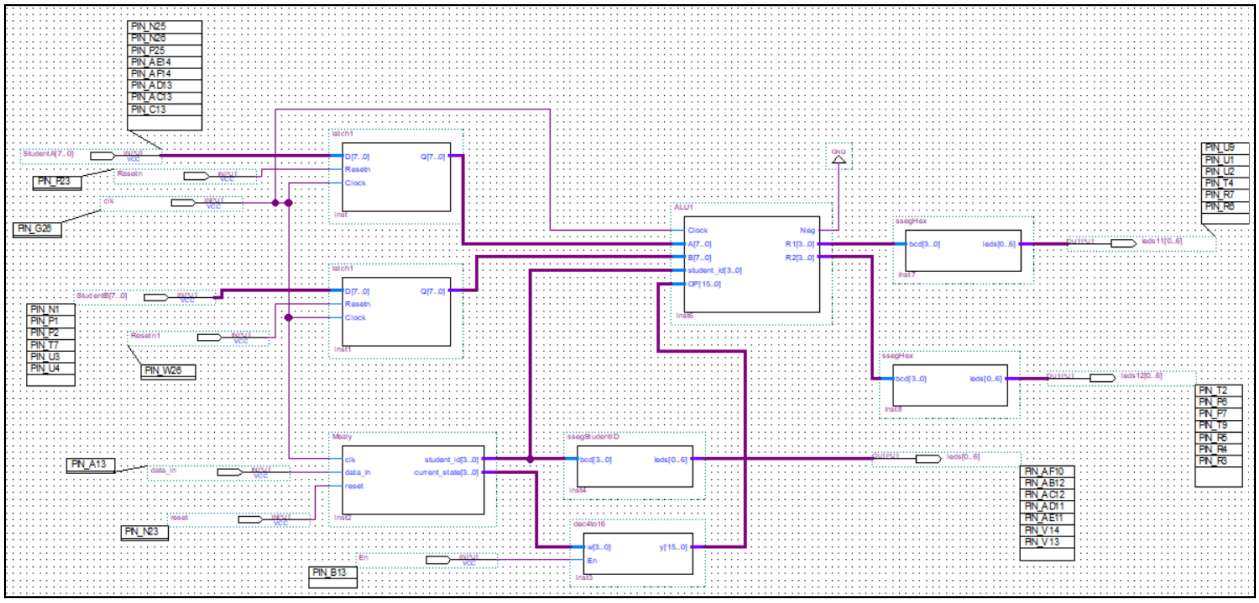
ALU 2 is a modified Arithmetic Logic Unit designed to execute complex data manipulation and routing tasks specified in Table A of the lab manual. While it shares the same exact external architecture with ALU1, its internal architecture has been reprogrammed to perform the operations specified in Table A. Instead of basic arithmetic, ALU 2 handles advanced bitwise operations such as logical shifts, bit rotations, finding the minimum between two values, and inverting bit-significance order. Likewise ALU1, the ALU2 also relies on the OP code from the 4x16 decoder to trigger specialized functions on the rising edge of the clock.

Block Diagram:



	Name	Alias	Inversion	Status	Direction
1	A[7..0]	A[7..0]	None	Used	INPUT
2	B[7..0]	B[7..0]	None	Used	INPUT
3	Clock	Clock	None	Used	INPUT
4	Neg	Neg	None	Used	OUTPUT
5	OP[15..0]	OP[15..0]	None	Used	INPUT
6	R1[3..0]	R1[3..0]	None	Used	OUTPUT
7	R2[3..0]	R2[3..0]	None	Used	OUTPUT
8	student_id[3..0]	student_id[3..0]	None	Used	INPUT

Circuit Schematics for Problem Set 2:



Inputs and Outputs of the ALU:

- *Clock (Input):* Global signal responsible for ALU calculations to occur on the positive edge cycle.

- *A & B (Inputs):* The 8-bit dataset supplied by Latch 1 and 2.
- *Student_Id (Input):* 4-bit signal routed from FSM to the ALU.
- *OP (Input):* 16-bit operation selector generated by 4x16 decoder. The position of "1" in the OP code tells the ALU which operation to perform.
- *R2 (Output):* The upper 4 bits (7 down to 4) of the final computation result.
- *R1 (Output):*The lower 4 bits (3 down to 0) of the final computation result.

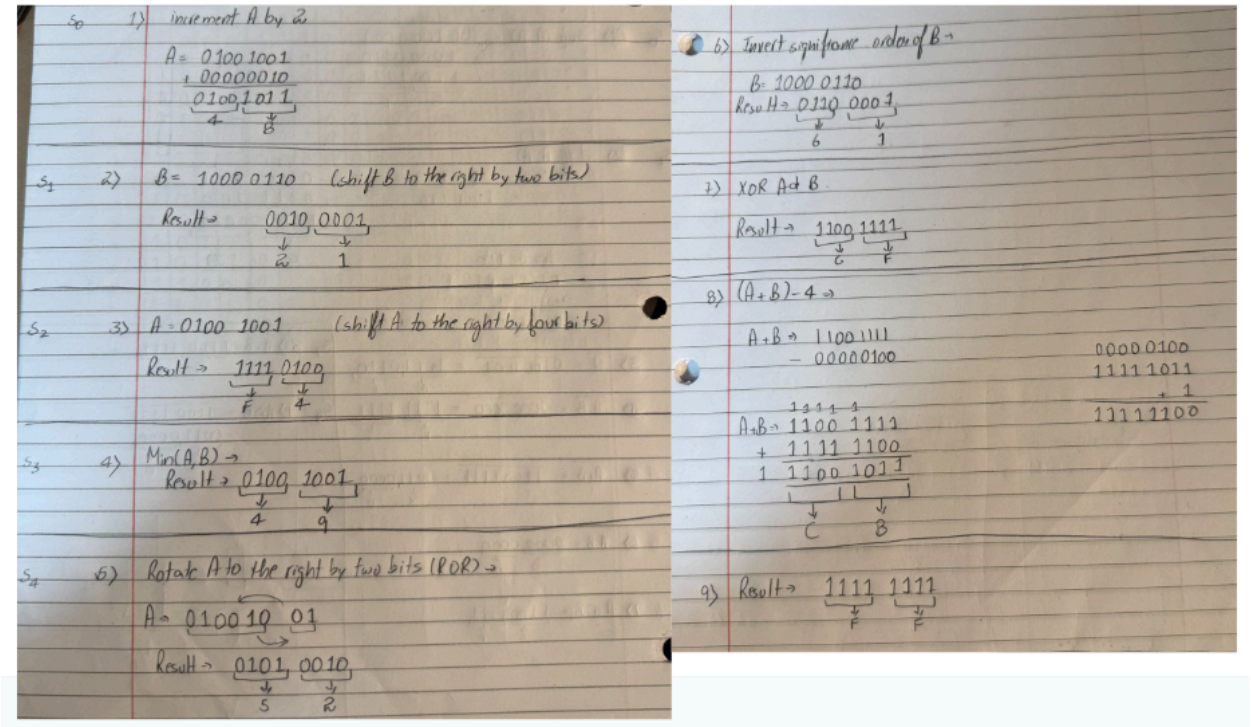
Table of function related to unique Microcode's generated by decoder for ALU_2:

a)

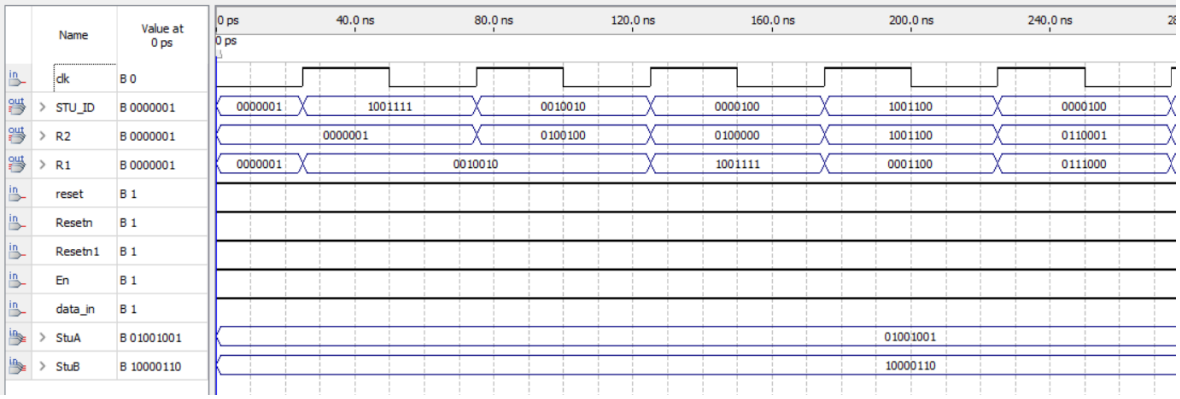
Function #	Operation / Function
1	Increment **A** by 2
2	Shift **B** to right by two bits, input bit = 0 (SHR)
3	Shift **A** to right by four bits, input bit = 1 (SHR)
4	Find the smaller value of **A** and **B** and produce the results (Min(**A**,**B**))
5	Rotate **A** to right by two bits (ROR)
6	Invert the bit-significance order of **B**
7	Produce the result of XORing **A** and **B**
8	Produce the summation of **A** and **B**, then decrease it by 4
9	Produce all high bits on the output

Calculations:

The following figure shows the hand calculations for each operation or function in Problem Set 2.

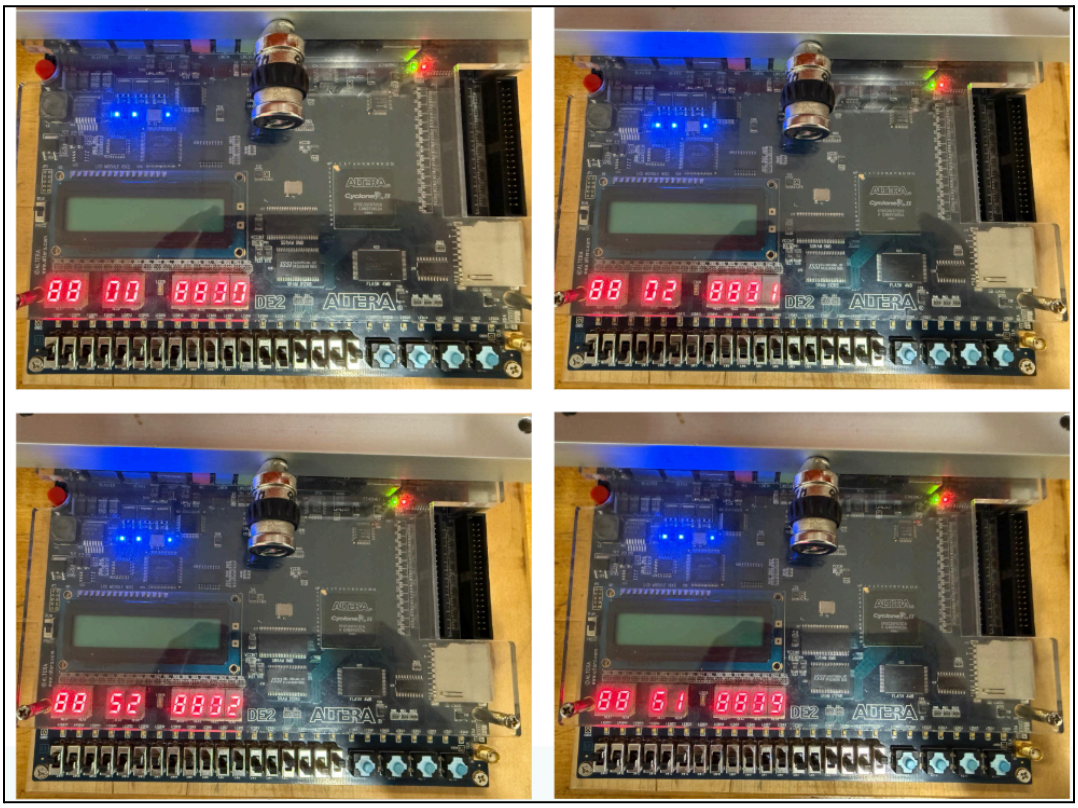


Waveform:



We can see here that the output R2 and R1 changes at every clock cycle. We set data_in to 1, in order to verify the waveform with the calculated results. When the first clock hits, the latches take the input and feed directly to the ALU. At the same time, the FSM outputs the state number to the decoder and the decoder converts it to OP code and feeds it to the ALU. Now, when the next cycle hits, ALU takes the input A and B from the latches, and OP code from the decoder and performs a specific function and outputs the results to R1 and R2. We can see in the waveform that when studentId is 1 and data_in is 1, the state number is 4, so when the next cycle clock hits, ALU should perform the calculator assigned to the OP code related to the state number 4. By hand calculation, we know that the result of R1 and R1 should be 5 2 which exactly matches in the waveform given above.

Verification of results on Altera DE2 FPGA board:

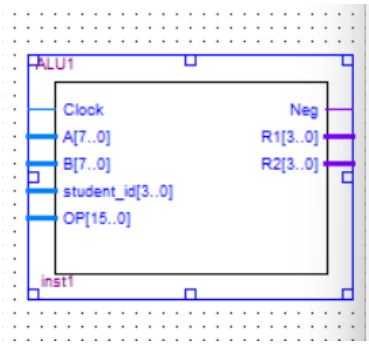


ALU_3 for Problem Set 3 of the Lab 6 procedure:

Description:

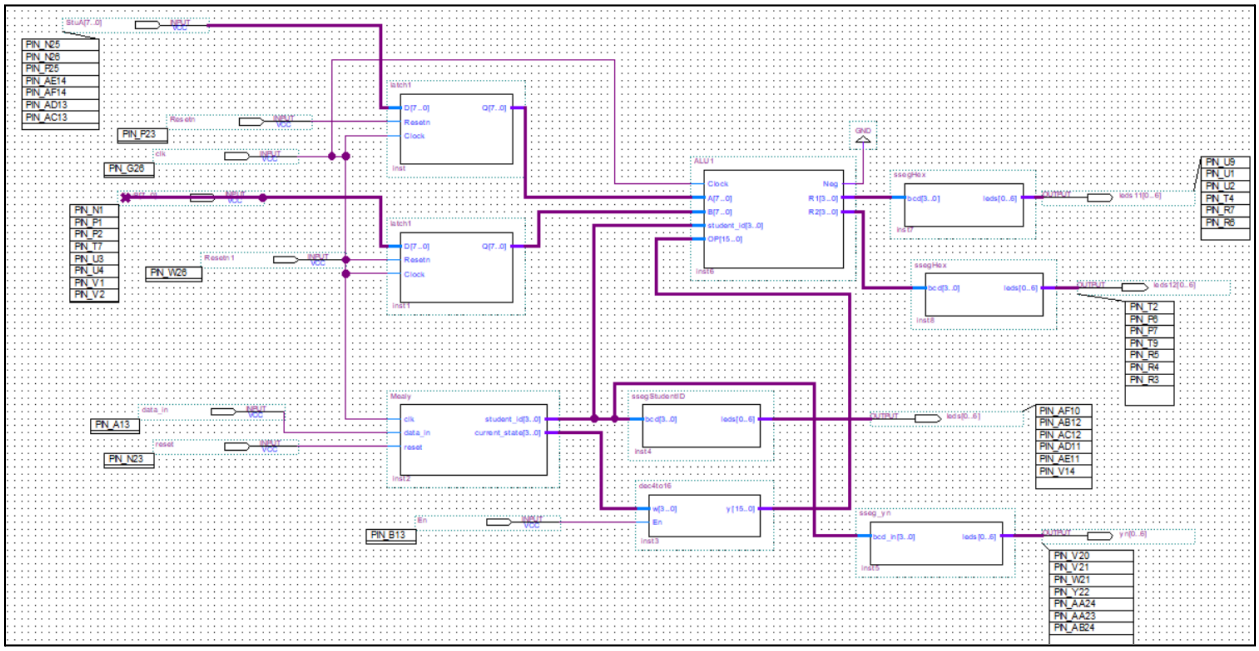
The internal and external architecture of the ALU 3 is exactly the same as that of ALU 1, but a new modified seven segment display is integrated with the Mealy FSM to solve the assigned problem from the problem set 3. My assigned problem from problem set 3 was to display “y” if the student_id is an odd number and display “n” if the student_id is an even number. For this, I created a separate seven segment file that checks if the last bit of 4-bit student_id is “1” or “0” in the FSM, and displays the “y” or “n” accordingly on a seven segment display**.**

Block Diagram:



	Name	Alias	Inversion	Status	Direction
1	A[7..0]	A[7..0]	None	Used	INPUT
2	B[7..0]	B[7..0]	None	Used	INPUT
3	Clock	Clock	None	Used	INPUT
4	Neg	Neg	None	Used	OUTPUT
5	OP[15..0]	OP[15..0]	None	Used	INPUT
6	R1[3..0]	R1[3..0]	None	Used	OUTPUT
7	R2[3..0]	R2[3..0]	None	Used	OUTPUT
8	student_id[3..0]	student_id[3..0]	None	Used	INPUT

Circuit Schematics for Problem Set 3:



Inputs and Outputs of the ALU:

- *Clock (Input):* Global signal responsible for ALU calculations to occur on the positive edge cycle.
- *A & B (Inputs):* The 8-bit dataset supplied by Latch 1 and 2.
- *Student_Id (Input):* 4-bit signal routed from FSM to the ALU.
- *OP (Input):* 16-bit operation selector generated by 4x16 decoder. The position of "1" in the OP code tells the ALU which operation to perform.
- *R2 (Output):* The upper 4 bits (7 down to 4) of the final computation result.
- *R1 (Output):*The lower 4 bits (3 down to 0) of the final computation result.
- *Leds (Output):* A 7-bit output routed to an additional seven segment display to output "1000100" to draw a "y" when the FSM student id is odd, and "1101010" to draw "n" when the FSM student id is even.

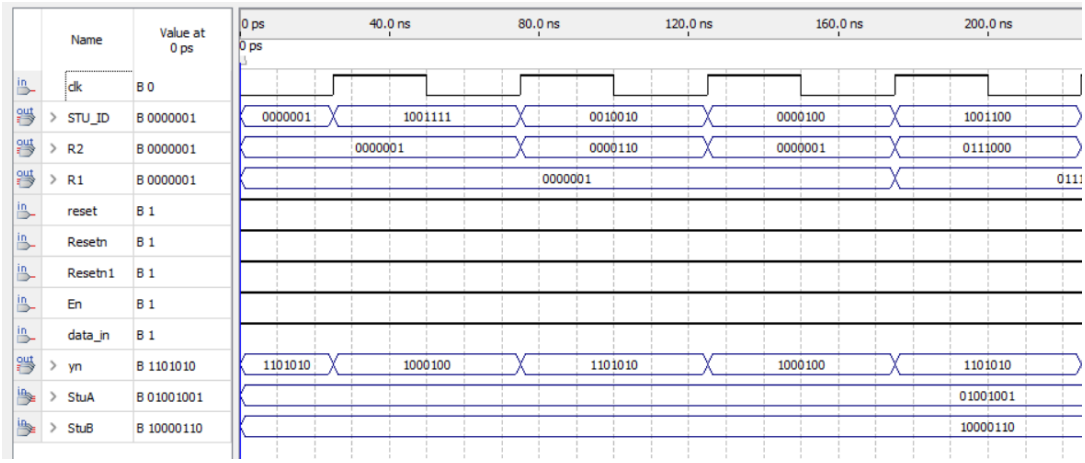
Assigned Problem from Problem Set 3:

a) For each microcode instruction, display 'y' if the FSM output (**student_id**) is odd and 'n' otherwise

Table of function related to unique Microcode's generated by decoder for ALU_3:

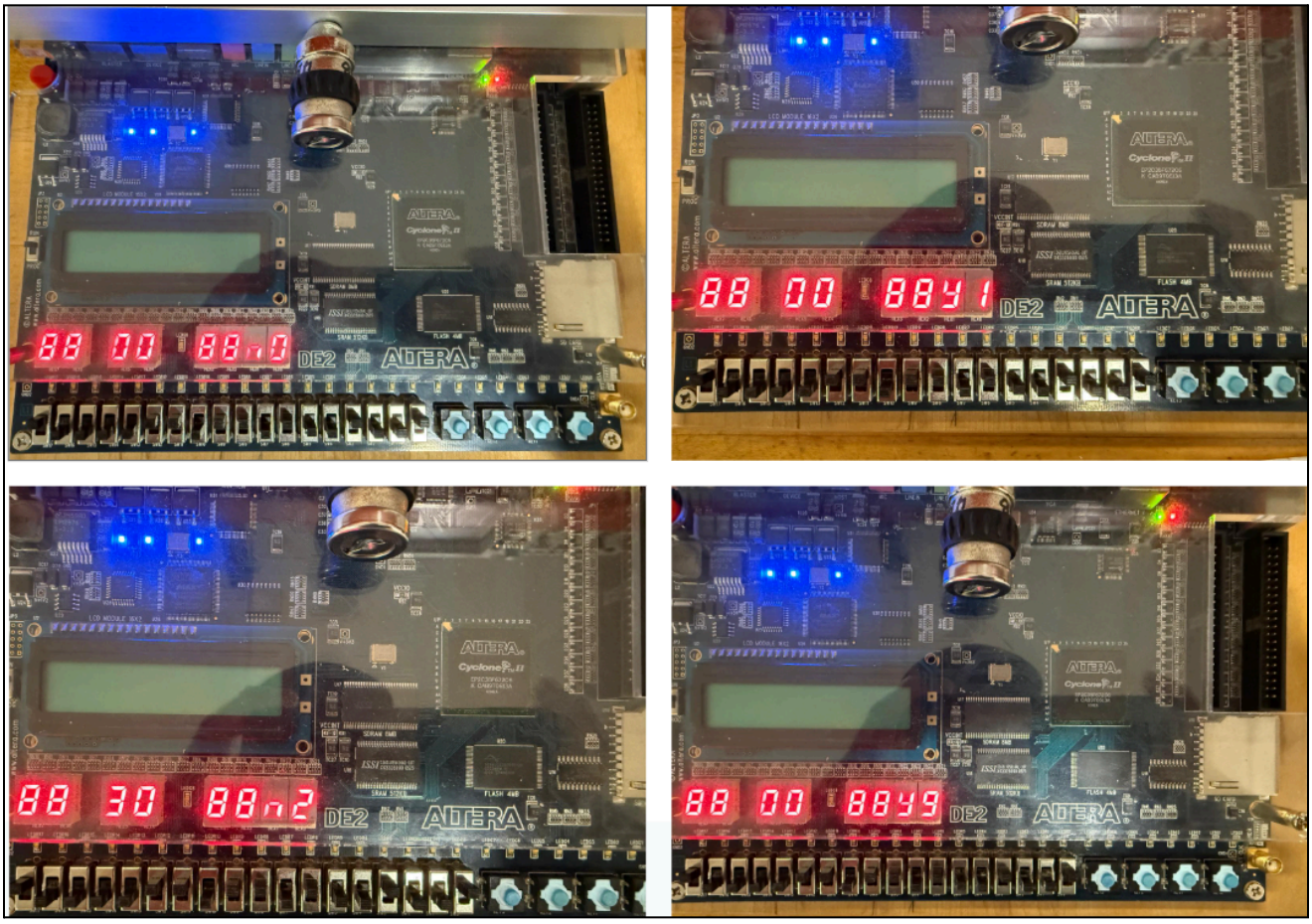
Function #	Microcode	Boolean Operation / Function
1	0000000000000001	sum(A, B)
2	0000000000000010	diff(A, B)
3	0000000000000100	$\overline{A}$
4	0000000000001000	$\overline{A \cdot B}$
5	0000000000010000	$\overline{A + B}$
6	0000000000100000	$A \cdot B$
7	0000000001000000	$A \oplus B$
8	0000000010000000	$A + B$
9	0000000100000000	$\overline{A \oplus B}$

Waveform:



Here in the waveform, we notice that the yn-seven segment display outputs “y” as “1000100” when student_id is an odd number and outputs “n” as “1101010” when student_id is an even number.

Verification of results on Altera DE2 FPGA board:



Conclusion:

This lab successfully demonstrated the design and implementation of a custom ALU unit which is controlled by a Mealy FSM. This lab verified the integration of several combinational and sequential components such as decoders, latches, and a central control unit to manage the datapath routing. We successfully tested and demonstrated the advanced bitwise operations, and validated signal fan-out by using a single FSM output to drive calculations and custom seven segment displays at the same time. Moreover, this lab helped bridging digital logic theory with the practical FPGA hardware implementation. I developed skills in debugging, coding, creating block diagrams as well as verifying the waveforms with manual hand calculations.